CAPSTONE / ARCHITECTURE

Multi-agent e-commerce support

Architecture plan v2

Version 2.0 | 2026-10-12 | America/New_York

LangChain + LangGraph + LangSmith + DeepEval

Evidence-grounded retrieval, optional provenance graph, independent policy enforcement, and simulated transactions.

MEASURED BASELINE / TARGET DESIGN

195 validated Phase 3 responses. Prompt selection remains open. Later capabilities require separate implementation and measured evidence.

This revision preserves the original nine-phase blueprint and incorporates the selected frameworks, evaluation approach, embedding strategy, and optional document knowledge graph. It is a design and delivery plan, not a claim that all target capabilities are deployed. The original architecture PDFs remain unchanged.

Design commitment

One bounded supervisor coordinates specialists. LangChain integrates language models, LangGraph expresses controlled workflow transitions, LangSmith provides opt-in observability, and DeepEval supports measured answer-quality evaluation. Independent deterministic services retain policy, authorization, confirmation, and action authority. Every transactional write is simulated.

Current evidence and open gates

Phases 1 and 2 have documented exit evidence. Phase 3 produced 195 schema-valid responses from the frozen 65 cases across prompts A/B/C with `qwen3.7-max`. The pre-framework offline suite recorded 51 passing tests. Those observations do not establish grounded answer accuracy, safe tool execution, or a default prompt. R05 conflict escalation, S15 threshold evidence, and M02 multi-intent coverage remain selection concerns. Phase 3 stays open; Phases 4-9 are not complete. The new framework integration has 75 passing tests and a 9/9 validated live smoke run. DeepEval, the catalog metadata graph, ETA rules, public-fact adapters, and Streamlit UI have initial verification described below. Hosted LangSmith and an embedding index remain unverified/unimplemented.

1. Revision decisions and change log

Area	Original plan	v2 decision
Model integration	Plain Python adapter initially	LangChain model integration with a stable application-facing contract
Orchestration	Framework optional	LangGraph for explicit bounded state transitions
Observability	Structured local logs; optional backend	Local sanitized logs plus opt-in metadata-safe LangSmith traces
Evaluation	Frozen harness and manual review	Preserve hard deterministic gates; add DeepEval with a separately configured judge
Generation	Configurable provider	Current `qwen3.7-max`; OpenAI or compatible endpoint via required URL and model name
Retrieval	Local FAISS and document chunks	Compare nano-only with asymmetric Voyage embeddings before selecting
Document relationships	Section metadata	Optional small provenance-backed knowledge graph for cross-document retrieval
Secret handling	Secrets outside source control	Ignored `.env`, key-free template, prepublication scanning; revoked key incident retained as a security lesson

LangGraph's workflow graph and a document knowledge graph serve different purposes. LangGraph nodes execute application steps. A knowledge graph stores domain relationships used to find evidence. Neither graph gives an LLM permission to execute an action.

2. Problem, scope, and frozen requirements

The primary user is an e-commerce customer. A support analyst receives sanitized handoffs when a request needs human authority. Policy documents explain what is allowed; verified order and carrier records establish the current state. Memory, retrieval scores, and generated explanations cannot substitute for either source.

Supported intents remain frozen: order status, shipping tracking, delivery estimate, return eligibility, refund request, cancellation, product information, warranty, escalation request, and bounded feedback style. Payment disputes, account management, coupons, legal demands, medical advice, and real customer-system writes remain outside scope. Safety complaints trigger escalation rather than diagnosis.

The local CLI remains the initial interface. An API can follow after the workflow stabilizes. All customers, policies, orders, tracking events, tickets, and action results are synthetic. Thresholds and retention rules come from explicit configuration rather than model assumptions.

The frozen dataset remains `data/evaluation_cases.json` v1.0: 65 cases comprising 20 normal, 15 retrieval/grounding, 15 safety/authorization, and 15 tools/memory/adaptation cases. The scenario date is fixed at 2026-10-12 for reproducible evidence. Additional cases require a dataset version bump; the original cases are not edited to improve scores.

3. Layers and framework responsibilities

Layer	Responsibilities	Boundary
Interaction	CLI/API input validation, session identity, presentation	No business-rule decisions or direct writes
Application control	LangGraph routing, budgets, state transitions, terminal outcomes	No unrestricted model-driven execution loop
Specialists	Order, returns, logistics, product, escalation interpretation	Narrow tools; proposals carry no authority
Domain services	Policy calculations, records, retrieval, memory, feedback	Framework-independent rules and typed contracts
Operations	LangChain provider adapter, LangSmith metadata, DeepEval, local logs	No secret or raw sensitive transcript export

LangChain: interchangeable model access

Require a configured API base URL and model identifier. The current measured provider alias is `qwen3.7-max`. OpenAI and compatible Chat Completions endpoints are supported through a provider adapter; no provider URL, key, or model name is silently guessed for a live experiment. Endpoint compatibility must be tested for structured output, usage metadata, timeouts, and supported parameters.

Use `langchain-openai` for standard OpenAI API semantics. Provider-specific extensions need an explicit adapter or integration; compatibility does not guarantee identical features. Keep typed response validation and bounded repair in the application. Generation and embedding configuration remain separate.

LangGraph: deterministic workflow

Target path: validate input -> safety gate -> classify intent -> bounded plan -> route specialists -> retrieve/call read tools -> validate evidence -> policy gate -> draft response or pending action -> verify outcome -> terminal response.

Conditional edges select refusal, clarification, insufficient evidence, escalation, pending confirmation, tool error, or success. Action execution is reachable only after deterministic authorization and exact confirmation. A missing or failed handoff tool cannot produce a completed escalation claim.

State carries request/session IDs, supported intents, concise plan summaries, evidence references, counters, policy decisions, pending action descriptors, errors, and terminal outcome. Do not persist private chain-of-thought. Persistence/checkpointing requires a separate retention and access design; framework availability alone does not establish durable memory.

Request budgets remain: at most 4 delegations, 8 tool calls, 2 retries, and 6 plan steps. Enforce counters independently of framework recursion limits. Budget exhaustion terminates safely. Repeated side effects require idempotency and reconciliation, never blind retry.

Specialist boundaries

Order specialists return verified state and permitted next steps. Returns specialists explain evidence and deterministic eligibility. Logistics specialists report the latest event and timestamp; estimates are not guarantees. Product specialists cite catalog facts and disclose missing specifications. Escalation specialists prepare sanitized handoff packets and report success only after a verified mock ticket result.

Simple UI and integration candidates

The user selected Streamlit with Python-authored application code and accepted its internal React dependency. The implemented clean interface has four tabs: Ask a question, Evaluation, Knowledge, and Delivery estimate. It presents responses, recorded evaluation runs, graph/provenance readiness, and manual or opt-in API delivery facts. Streamlit AppTest completed without exceptions, the baseline form worked, and a screenshot of the local app at `127.0.0.1:8501` was visually reviewed. These checks do not prove future transactional workflows. The CLI stays available for reproducible runs. The UI invokes application contracts and never bypasses policy or confirmation gates.

External demonstration candidates are DummyJSON for synthetic products, Mockoon for local order/tracking/failure APIs, and self-hosted Medusa Store API for a later commerce adapter. These are not connected. Prefer the existing versioned local fixtures for evaluation so upstream changes cannot alter frozen results. External adapters require schema validation, timeouts, explicit permissions, and simulated-write boundaries; API availability does not grant action authority.

Location-based delivery estimate

The user requested expected delivery based on location as a concrete logistics use case. Propose an `estimate_delivery` calculation service using destination city/postal code, verified dispatch status/date, shipping service, and configured zone/transit range. Avoid collecting a full street address for this demo. Return an estimated date interval, source references, rule/config version, and uncertainty; never guarantee delivery.

The user selected deterministic rules over externally supplied facts. For the separately versioned synthetic demo, base transit days are `max(1, ceil(distance_miles * 3 / 1000))`: 1000 miles implies 3 days. A storm or snow

disruption adds 2 days once, even when both labels describe the same event. The upper estimate adds a configurable 2-day buffer. These are transparent demo rules, not carrier service commitments or measured delivery predictions.

Rule precedence: validate non-negative distance and relevant dated weather facts; establish a verified dispatch/date anchor; apply base transit; add the single disruption delay; apply the uncertainty buffer; return the interval with every contributing source and rule version. Missing distance, weather evidence, or dispatch information produces insufficient evidence rather than silently assuming favorable conditions. A dispatched parcel uses verified dispatch date; a processing order needs a verified/planned dispatch assumption before an ETA can be calculated. Evaluate boundary distances, duplicate disruption labels, absent facts, and uncertainty explicitly.

Initial allowlisted read-only fact adapters use OSRM for route distance and National Weather Service public alerts. An opt-in city-center New York-to-Denver demo returned 1781.28 miles and zero active destination alerts. Zero alerts does not mean a clear-weather forecast; destination alerts do not cover the entire route. OpenRouteService or Open-Meteo are possible later alternatives. Route distance is a proxy rather than the actual carrier route; weather relevance must be bounded by route/location and dates. External facts inform the deterministic rules and cannot replace verified carrier events or retailer shipping policy. Manual facts remain available in the UI with explicit provenance; API lookup is opt-in.

Carrier events remain authoritative for tracking. Distance, routing, or weather APIs may add context but cannot establish parcel arrival alone. Unknown dispatch state or missing transit rules produces clarification or insufficient evidence, not a guessed date. An ETA form can expose the minimal inputs. Keep a separately versioned synthetic ETA demo/configuration; do not silently alter the frozen 65 cases or original policy thresholds. This is a proposed extension contract beyond the original eight-tool inventory and needs its own validation before execution claims.

4. Knowledge, vector retrieval, and optional graph

Corpus preparation comes first

The planned seven documents are `return_policy.md`, `refund_policy.md`, `shipping_policy.md`, `cancellation_policy.md`, `warranty_policy.md`, `faq.md`, and `escalation_guidelines.md`. Their inventory is frozen, but the authoritative corpus is not yet authored and indexed. Structured sources remain `product_catalog.json`, `sample_orders.json`, `sample_tracking_events.json`, and `policy_rules.json`.

Required document inventory for the extended corpus

The following inventory incorporates the user's later document requirements. Preserve the original seven-document baseline and identify additions as a versioned corpus extension; new content does not silently change supported intents or frozen test expectations. These documents are planned and not yet fully authored/validated.

Document family	Required content
Product catalog and guides	Product/category IDs, specifications, compatibility, sizing, care, exclusions, and guide references
Return policy	Windows, condition/category restrictions, date calculation, required evidence, exceptions
Refund policy	Thresholds, approved channels, time ranges, escalation requirements, simulated-action boundaries

Document family	Required content
Shipping policy	Shipping services, zones, dispatch assumptions, tracking interpretation, estimates and limitations
Delivery estimate rules	Versioned distance/weather calculation, date anchor, buffers, missing-data behavior, uncertainty
Cancellation policy	Order/dispatch-state eligibility, exact confirmation, rejected and unknown outcomes
Warranty policy	Covered categories, exclusions, evidence requirements, human-review route
Account-help guidance	Informational troubleshooting and official support routes; no account access or automated changes
Escalation guidelines	Injury, thresholds, policy conflict, queue/priority, sanitized handoff and failure fallback
Privacy and security	Minimization, secrets, trace export, retention/reset/deletion, access boundaries
FAQ	Common questions with links to authoritative policy sections; no duplicate independent policy authority

Every document requires a stable document ID, version, effective date, section IDs, owner, authority level, and explicit supersedes/relationship links where applicable. Preserve provenance from each chunk and graph edge to these fields. Structured product and operational records retain their own schema/version and are linked rather than rewritten as policy. An FAQ or product guide cannot override an authoritative policy section. Missing owners/effective dates or ambiguous precedence are ingestion defects requiring review.

Account-help remains informational guidance only. Account/login management and automated account changes remain outside the frozen v1 scope until an explicit versioned scope extension is approved and evaluated.

Ingestion preserves document ID, version, section ID, effective date, category applicability, content hash, and source location. Start with section-aware chunks around 400-700 tokens and 50-100 tokens overlap, then tune using independently labeled retrieval cases. Changing chunking, corpus, or embeddings creates a new index version. Quarantine malformed documents; untrusted document instructions remain data.

Embedding candidates

Candidate A is a local nano-only baseline: `voyageai/voyage-4-nano` for documents and queries. Candidate B uses hosted `voyage-4-large` for document embeddings and local `voyageai/voyage-4-nano` for query embeddings. Voyage documents the shared 4-series embedding space; verify the actual local revision and API model before mixing vectors.

Both sides explicitly produce 1024 dimensions with consistent normalization and precision. Use correct query/document input modes and retrieval prefixes. Validate dimension and finite values at ingestion/query time. Pin the local model revision and dependencies; retain model IDs, dimension, normalization, chunking, and hashes in the index manifest. A mismatch fails closed rather than querying an incompatible index.

The local model has not been installed. Asymmetric indexing introduces a hosted embedding dependency and document export to Voyage; use only approved synthetic corpus content and a separate credential. Do not assume cost or accuracy improvements before measuring this small corpus. Compare Recall@5, citation support,

latency, resource usage, and available token/cost evidence. Select thresholds from answerable/unanswerable examples rather than an arbitrary similarity cutoff.

Why a knowledge graph can help

Vector search handles paraphrases well. A small graph can connect a product to its category, a category to applicable restrictions, and a policy section to its document version. It is useful when an answer depends on relations spread across documents. It is optional for this small corpus and must earn its complexity through comparison with vector-only retrieval.

Start with local typed adjacency records or SQLite, not a separate graph database. Initial node types: Product, Category, PolicyDocument, PolicySection, and DocumentVersion. Initial edges: `belongs_to`, `applies_to`, `contains`, `version_of`, and `supersedes`. Every edge stores a stable ID, source document/section/version, effective date where available, ingestion version, and provenance. Reject missing endpoints and unsupported relations. Prefer deterministic extraction from curated metadata; any LLM-proposed edge is provisional until validated.

The initial implementation contains product/category metadata relationships from the existing synthetic catalog. It contains no order/customer graph; adding those relationships needs explicit authorization and access boundaries. Add policy section/version links only after Phase 4 authors the corpus. A fixture graph does not supply missing authoritative policy evidence to the Phase 3 comparison.

Example: TrailRunner Shoes -> footwear -> return-policy footwear section -> policy version. The graph finds the applicable section; a deterministic policy service still verifies delivery date and eligibility. A link saying a policy applies is not proof that the customer qualifies.

Hybrid retrieval algorithm

1. Resolve product/category references from verified catalog data; clarify ambiguity.

2. Retrieve vector candidates with source and version filters.

3. Expand allowlisted graph relations by at most two hops within a bounded candidate budget.

4. Merge and deduplicate by source/version/section; validate effective dates and provenance.

5. Rerank/select evidence within a context budget; retain vector versus graph contribution.

6. Surface contradictions explicitly. Abstain or escalate when sources conflict or are missing.

Compare vector-only and hybrid runs on the same corpus and labels, including category exclusions, superseded policies, missing edges, incorrect entity links, and conflicting versions. Measure retrieval recall and grounded answer quality separately. Full Microsoft GraphRAG community summaries and broad entity extraction are deferred unless corpus scale and measured needs justify them. The graph cannot authorize returns, refunds, cancellation, or record modification.

5. Tools, contracts, and transactions

Proposed tool	Access	Purpose
`search_policy`	Read	Authoritative policy passages
`lookup_order`	Read	Synthetic order state
`get_tracking_events`	Read	Carrier events and timestamps
`get_product_details`	Read	Catalog facts
`check_return_eligibility`	Calculation	Apply deterministic rules
`create_return_request`	Simulated write	Gated return request
`create_escalation`	Controlled mock write	Verified handoff ticket
`search_web`	Optional external read	General context; never retailer-policy authority

Each tool requires typed input/output, access category, timeout, error contract, execution ID, permission checks, and retry policy. Agent results include agent name, status, summary, resolvable evidence references, proposed action, and escalation reason. Reject unknown statuses, unresolved citations, and malformed arguments. Generated tool calls cannot import code, run shell commands, access arbitrary files, or expand permissions.

Cancellation/refund demos require additional action-service contracts; the eight-tool inventory does not implement them. Define these contracts in Phase 5 before claiming their demos work. Preserve denied, failed, and unknown-outcome behavior explicitly.

The action state machine remains Proposed -> Validated -> Awaiting confirmation -> Executing -> Completed / Failed / Unknown outcome, with Rejected, Cancelled, Expired, and Human review exits. Bind confirmation to session, action ID, action type, exact parameters, and expiry. Revalidate policy and current record state at execution. Verify returned state before claiming success. A timeout after execution may have started requires reconciliation before retry.

6. Evaluation with DeepEval

DeepEval is the selected quality-evaluation library. The frozen harness remains the experiment runner and evidence ledger. DeepEval complements deterministic tests; it never authorizes actions or overrides a hard safety failure.

Build test cases from actual input, actual output, expected behavior, authoritative retrieval context, and tool traces where available. Compare A/B/C under fixed model, evidence, dataset, decoding, and tool availability. Store case IDs, hashes, timestamps, returned model ID, latency, usage, evaluator configuration, scores, reasons, and errors.

Use answer relevancy and faithfulness metrics where the necessary context exists. Evaluate contextual relevance/recall only with independently reviewed relevance labels and expected evidence. Use a custom rubric for escalation and multi-intent completeness where appropriate. Phase 3 empty-context responses cannot establish RAG faithfulness or retrieval recall. Missing context, judge configuration, or evaluator failures are reported as skipped/error, never converted into passing scores.

A judge model is separately configurable and may use another OpenAI-compatible provider. Record its URL alias, model ID, rubric version, temperature, and costs where available. Avoid treating self-evaluation by the generation model as independent evidence. LLM judge outcomes are probabilistic and require calibration against human review; exact safety outcomes use assertions.

Hard gates: any failure blocks acceptance

Gate	Required evidence
No prohibited/unconfirmed simulated action	Zero violations in deterministic/adversarial suite
No secret/PII leakage in logs	Zero detected leaks in canary scans
Safe injected failures	100% safe handling; no false success
Mandatory escalation	All mandatory cases, including S01, S15, R05
No unsupported policy facts	Claim/source review; safe abstention when missing

Quality targets remain unchanged

Grounded answer accuracy >=95%; escalation precision >=90% and recall >=95%; tool selection >=95%; retrieval Recall@5 >=90%; citation support >=95%; ordinary structured-output validity >=99%; supported completion >=95%; non-tool single-agent p95 latency <=8 seconds. Targets are not achieved merely by installing frameworks or passing schema validation. Hard mandatory escalation remains 100% regardless of aggregate recall.

Recorded Phase 3 evidence

Final run `20261013T030651Z-b48a462e` used 65 cases x 3 prompts, temperature 0, 1200 completion tokens, thinking disabled, empty evidence, no tools, and no memory. All 195 final responses validated. A/B/C median latency was 6.126/6.142/5.155 seconds; p95 was 7.967/7.841/6.249. C needed one repair. Total recorded tokens: 175,357. Cost was not calculated. Concurrent experiment latency does not establish the final single-agent performance gate.

Selected-case assistant review found unsupported return-window suggestions in A/B, R05 escalation failure in C, missing S15 threshold evidence, and omitted feedback intent in M02 for C. These are qualitative observations, not a full-suite accuracy score. No default prompt is selected. Tighten conflict-escalation and multi-intent handling; provide authoritative threshold evidence in the appropriate phase; rerun and preserve before/after artifacts. Keep failures in the evidence ledger.

Framework integration verification addendum - 2026-10-13

The new LangChain client and actual LangGraph prepare -> model -> validate -> bounded repair -> finalize workflow passed a live N01/R05/M01 smoke comparison across A/B/C: 9/9 responses validated, recorded under `artifacts/prompt_runs/20261013T034118Z-745df983`. The offline suite now passes 75 tests, including a judge-failure privacy regression. This smoke run verifies integration rather than replacing the frozen full comparison or satisfying future tool/RAG gates.

DeepEval deterministic metrics evaluated the saved 195 outputs: 195 contract passes and 7 mandatory-escalation classification failures. Schema success is therefore not semantic approval. An optional

GEval judge adapter smoke run on N01 A/B/C produced three scores of 1.0 using the same Qwen model; these are adapter smoke results, not independent judge evidence or full-suite answer quality. A separate calibrated judge remains recommended.

The metadata-only LangSmith wrapper is implemented and privacy-filter tests pass. No LangSmith API credential is configured, so hosted trace ingestion remains unverified. The lightweight catalog graph, deterministic ETA rule engine, and four-tab UI have initial local verification. The extended policy corpus is still planned, not fully authored/indexed; no measured vector/hybrid RAG results are claimed.

7. LangSmith and privacy boundaries

LangSmith is the selected observability backend, enabled explicitly with local credentials. The safe default keeps remote tracing disabled. Send allowlisted metadata: correlation ID, model/prompt/index/dataset versions, stage duration, token counts, counters, outcome/error category, source IDs, and tool names/statuses. Do not send keys, raw customer messages, model outputs, record payloads, full prompts, or chain-of-thought by default.

Avoid automatic LangChain callbacks that export entire inputs/outputs. Wrap explicit sanitized spans and test with synthetic secret/PII canaries. Validate both local artifacts and outbound trace payloads. If remote tracing is unavailable, continue with local sanitized logs; observability failure must not alter a policy decision. Live LangSmith connectivity remains unverified until credentials and a controlled metadata-only trace are tested.

Credentials belong only in ignored owner-readable `.env`. `.env.example` contains names and placeholders, never keys. The user reports revoking the previously committed key; this mitigates its future use but does not remove historical exposure. Scan proposed commits/artifacts before publication, including templates and evaluation outputs. Do not echo secrets in errors. Repository-history cleanup is a separate operation requiring explicit coordination.

DeepEval runs locally by default. Judge calls and LangSmith exports cross separate data boundaries; configure them explicitly and minimize payloads. Do not enable Confident AI dataset/result uploads implicitly. Synthetic evaluation content can be published only within the user's approved scope after sanitization. Real customer data requires a separate access, retention, deletion, and export policy.

8. Nine-phase delivery roadmap

Phase 1 - Problem and success definition

Preserve frozen personas, scope, intents, source inventory, synthetic fixtures, 65-case dataset, and acceptance criteria. Exit evidence exists. Any requirement change gets a version and review rather than silently changing the benchmark.

Phase 2 - Rule-based baseline

Preserve modular CLI, deterministic templates, reproducible run logs, and documented paraphrase/multi-intent limitations. Exit evidence exists. Baseline remains available for comparison after framework integration.

Phase 3 - LLM and prompt evaluation

The LangChain adapter and bounded LangGraph response workflow are implemented with smoke evidence. DeepEval deterministic metrics and optional judge adapter are initially verified; metadata-safe LangSmith spans remain locally tested with hosted ingestion unverified. Retain typed outputs and controlled A/B/C runs. Rerun selection concerns; exit only after a documented gate-based default selection. Status: open despite successful framework integration.

Phase 4 - Knowledge and retrieval

Author versioned synthetic policies; ingest sections/chunks; install and pin candidate embeddings; build versioned FAISS index; evaluate nano-only versus asymmetric retrieval. Add lightweight graph as an optional ablation after vector baseline. Compare LLM-only, vector RAG, and hybrid RAG. Exit: measured retrieval quality and citation examples, with missing/conflicting evidence handled safely.

Phase 5 - Tools and specialist orchestration

Implement eight allowlisted typed tools, central routing, five specialists, permissions, policy gates, and simulated action contracts. Define cancellation/refund actions separately. Test malformed inputs, nonexistent records/tools, timeouts, denied actions, and loop budgets. Exit: bounded multi-step workflows and forced negative tool test with no unauthorized action.

Phase 6 - Planning, memory, and context

Add explicit plans, permitted session context, TTL, reset/deletion, and conflict handling. Prefer short-lived structured state; store preferences only when explicitly approved. Test follow-ups and context switches. Current records and policies always supersede memory. Exit: measured multi-turn improvement and reset evidence.

Phase 7 - Adaptive behavior

Collect controlled helpful/not-helpful/too-verbose/too-short feedback. Change only allowlisted style/retrieval preferences with minimum samples, validation, versioning, and rollback. Safety, thresholds, permissions, and confirmation are immutable to feedback. Exit: reproducible before/after improvement without safety regression.

Phase 8 - Deployment readiness

Pin runtime/dependencies; document startup, ingestion, indexing, evaluation, and tracing commands. Validate required configuration without exposing values. Measure latency/errors and scan canaries. Another person must run the app from documented commands in a clean local venv. Exit: reproducible operability, safe failures, and observability evidence.

Phase 9 - Evaluation and engineering review

Run all frozen cases and hard gates on the final build. Report measured quality, retrieval, tools, memory, adaptation, escalation, latency, and errors. Include a defect's cause, fix, permanent regression test, before/after result, and residual risk. Exit: rubric evidence, forced demos, disclosed limitations, and prioritized roadmap.

9. Forced demos and required artifacts

Preserve five interactions: grounded shoes-return eligibility; delayed package plus refund multi-agent coordination; cancellation with exact confirmation; injury/outside-window escalation; feedback plus multi-turn memory and reset. Add a nonexistent tool or injected timeout as the negative demo. No demo is marked passed from its expected behavior alone.

Each evidence packet records case ID, app/prompt/corpus/index/dataset versions, sanitized stage trace, source references, validated tool results, final response, expected outcome, pass/fail, and reviewer notes. Screenshots illustrate behavior; logs and test outputs establish correctness. Preserve complete comparison outputs and failed diagnostics with sanitized content.

Repository targets remain modular: `application/`, `orchestration/`, `agents/`, `domain/`, `retrieval/`, `tools/`, `safety/`, `memory/`, `feedback/`, `prompts/`, `observability/`, and `evaluation/`. Framework code stays at integration boundaries; business rules stay testable without network services. Final deliverables include working agent, problem framing, demos, prompt comparison, evaluation report, and engineering justification.

10. Risks, next steps, and references

Main risks are framework complexity, metadata leakage, unsupported policy claims, graph relation mistakes, embedding mismatch, judge bias, and averages hiding safety failures. Mitigate with narrow adapters, explicit trace payloads, independent policy enforcement, provenance validation, versioned manifests, calibrated review, and binary gates.

Immediate work: configure an independent optional judge and LangSmith safely; repair prompt-selection issues; author the Phase 4 corpus; establish vector-only retrieval before adding policy graph expansion. Keep the verified catalog graph distinct from future hybrid RAG. Do not infer future capabilities from dependency installation or a smoke run.

Primary technical references

- LangChain ChatOpenAI integration: https://docs.langchain.com/oss/python/integrations/chat/openai

- LangGraph overview: https://docs.langchain.com/oss/python/langgraph/overview

- LangSmith observability: https://docs.langchain.com/langsmith/observability

- DeepEval metrics: https://deepeval.com/docs/metrics-introduction

- Voyage embeddings: https://docs.voyageai.com/docs/embeddings

- Voyage nano model card: https://huggingface.co/voyageai/voyage-4-nano

- Voyage shared-space announcement: https://blog.voyageai.com/2026/01/15/voyage-4/

Project evidence: `docs/requirements.md`, `docs/acceptance_criteria.md`, `docs/phase3_live_results.md`, `docs/embedding_strategy.md`, `docs/safety_and_privacy.md`, and the unchanged original `docs/capstone_architecture_plan.pdf`. External references describe capabilities; local artifacts establish what this project has measured.